

Project Report: Dynamic Timetable Generator

1. Project Title

Dynamic Timetable Generator: An Automated Constraint-Aware Academic and Personal Scheduling System

2. Executive Overview

Academic institutions face recurring operational challenges each semester when compiling master timetables. Allocating resources manually involves mapping overlapping variables: instructors, classroom sizes, specialized facilities, and student cohorts, while preventing scheduling conflicts.

The **Dynamic Timetable Generator** is a web-based scheduling platform engineered to automate resource distribution. Implemented using Python, Flask, and an interactive Tailwind CSS interface, the application applies a randomized heuristic optimization algorithm to allocate courses into designated timeslots without instructor overlaps, room overcrowding, or student cohort clashes. The system features real-time conflict tracking, live utilization analytics, a streamlined "Student Mode" for individual study planning, and direct client-side export pipelines to PDF and CSV formats.

3. Project Objectives

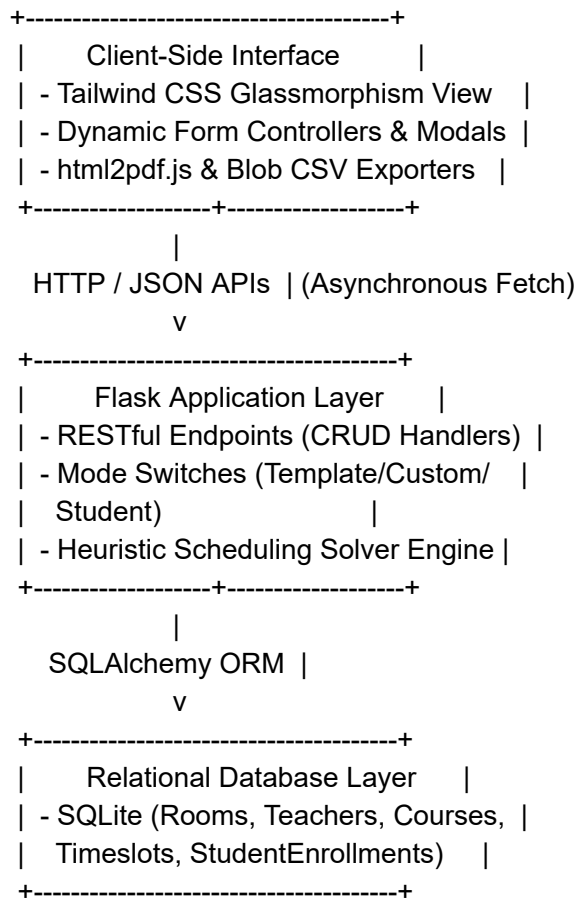
- **Automate Academic Scheduling:** Eliminate manual spreadsheet assembly by mapping course requirements to available institutional resources using constraint satisfaction techniques.
- **Enforce Complex Constraints:** Implement validation rules to prevent:
 - Double-booking of instructors or physical rooms.
 - Exceeding instructor daily workload limits (max_hours_per_day).
 - Assigning lectures to rooms with insufficient capacity or incorrect room types (e.g., placing laboratory coursework in lecture halls).
 - Overlapping schedules for students enrolled across multiple courses.
- **Graceful Conflict Handling:** Implement a heuristic engine that does not crash when presented with over-constrained data, but instead schedules all valid classes and

highlights unresolvable bottlenecks for administrative review.

- **Provide Dual Operating Modalities:** Support both an institutional management framework and a personal "Student Mode" designed for individual study planning.
- **Deliver Real-Time Usability:** Present an accessible, dark-mode Glassmorphism interface complete with live utilization metrics and one-click PDF/CSV document exports.

4. Technical Architecture

4.1 System Architecture Diagram



4.2 Architectural Layers

- **Presentation Layer:** Built as a single-page interface using HTML5, Tailwind CSS, FontAwesome icons, and Vanilla JavaScript. The layout features interactive modal dialogs, cursor illumination effects, dynamic toast notifications, and client-side document processing via html2pdf.js.
- **Application / REST API Layer:** Implemented in Flask, providing routes for institutional resource management (/api/room, /api/teacher, /api/course, /api/timeslot,

/api/enrollment) and lifecycle controls (/api/setup_dummy, /api/reset, /api/setup_student, /api/generate).

- **Optimization Engine:** The heuristic algorithm parses relational entities, enforces operational constraints, allocates slots, and calculates analytical metrics.
- **Data Persistence Layer:** Managed via SQLite using Flask-SQLAlchemy. Entity schemas include:
 - **Room:** id, name, capacity, room_type.
 - **Teacher:** id, name, max_hours_per_day.
 - **Course:** id, code, enrolled_count, weekly_hours, room_type, teacher_id.
 - **Timeslot:** id, name (formatted with start and end times).
 - **StudentEnrollment:** id, student_id, course_id.

5. Methodology & Algorithmic Implementation

5.1 Algorithmic Design

The platform uses a **Randomized Heuristic Optimization Algorithm** operating under a greedy assignment strategy:

- 1. Fetch all Courses, Rooms, Teachers, Timeslots, and StudentEnrollments from database.
- 2. Construct fast in-memory lookup maps for student-to-course and course-to-student associations.
- 3. Sort Courses in descending order based on class size / enrolled count (allocating hardest-to-place courses first).
- 4. For each Course:

While hours_scheduled < course.weekly_hours AND attempts < MAX_ATTEMPTS (300):

- a. Pick a candidate Timeslot (t) and Room (r) uniformly at random.
- b. Extract day of the week from Timeslot string.
- c. Check Hard Constraints:
 - Is Room type equal to Course room type? (Yes/No)
 - Is Room capacity >= Course enrolled count? (Yes/No)
 - Is (t, r) already occupied in the timetable? (Yes/No)
 - If Course has a teacher:
 - * Is (t, teacher) already booked? (Yes/No)
 - * Has teacher reached max_hours_per_day for this day? (Yes/No)
 - For all enrolled students in Course:

* Is (t, student) booked in another course? (Yes/No)

d. If all checks pass:

- Record assignment: $\text{timetable}[(t, r)] = \text{course.id}$
- Update teacher workload and student schedules
- Append entry to schedule output
- Increment hours_scheduled

If $\text{hours_scheduled} < \text{course.weekly_hours}$:

- Append Course to conflicts list

- 5. Compute optimization metrics (Total Classes, Room Utilization %, Empty Slots).
- 6. Return sorted JSON payload containing schedule, metrics, and conflict logs.

5.2 Constraint Verification & Conflict Reporting

Rather than halting execution upon encountering conflicting constraints (such as a 90-student class requiring a laboratory when the largest available lab accommodates only 80 students), the solver assigns all viable courses and records unassigned hours in a conflicts array. The UI alerts the administrator with a specific warning toast detailing which course could not be accommodated, allowing for administrative adjustments.

5.3 Modality Configurations

- **Institutional Mode:** Enables full CRUD interactions across all five database models, allowing institutions to generate comprehensive faculty and room schedules.
- **Student Mode:** Seeds a virtual room (Personal Workspace) and a virtual instructor (Self Study), hiding institutional parameters from the UI and presenting the user with a streamlined, two-panel subject-and-timeslot planner.

6. Optimization Metrics & Output Handling

The platform evaluates the efficiency of generated schedules using three primary metrics:

- **Total Classes Scheduled (N_c):** Count of successful slot allocations.
- **Total Available Slots (S_{total}):** Calculated as $|\text{Timeslots}| \times |\text{Rooms}|$.
- **Room Utilization Rate (U_r):**

$$U_r = \left(\frac{N_c}{S_{total}} \right) \times 100$$

- **Empty Slots:** Calculated as $S_{total} - N_c$.

Final timetables can be exported client-side as landscape PDF documents styled for printing or saved as CSV files for spreadsheet applications.

7. Future Scope & Enhancements

- **Integer Linear Programming (ILP) Integration:** Incorporate hybrid solvers (such as Google OR-Tools CP-SAT) alongside the heuristic engine to support formal mathematical optimization for large datasets.
- **Multi-Week and Bi-Weekly Timetabling:** Expand the timeslot schema to support rotating class schedules, alternating lab sessions, and weekend schedules.
- **User Authentication & Role-Based Access Control (RBAC):** Introduce dedicated user roles (e.g., Institutional Admin, Faculty Member, Student) to enable personal schedule tracking and self-service timetable viewing.
- **Calendar Synchronization:** Integrate Google Calendar and Outlook Calendar APIs (.ics generation) so faculty and students can sync schedules directly to mobile and desktop clients.
- **Soft Constraint Scoring:** Implement penalty-based optimization weights (such as avoiding back-to-back lectures for instructors or minimizing split schedules for student cohorts).